STRIPE PROJECT

1. OLTP Data System

D2. OLTP Entity-Relation Diagrams

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D2 : OLTP ERD / v36 --- 2026/11/05.*

D2.1. Contexte

Ce document présente les diagrammes dits *ERD* (*Entity-Relation Diagram*) du modèle de données *OLTP*. Il s'agit de représenter les entités et les relations du modèle *logique*. Les éléments de construction du modèle *conceptuel* sont données en annexe A [D2-A]. La définition de référence du modèle *physique*, avec typages précis, contraintes et indexations, est donnée en annexe B [D2-B]. Les détails de conception (justification, etc.) sont donnés en annexe C [D2-C]. Une stratégie de réplication et de recouvrement est proposée en annexe D [D2-D] pour répondre à l'exigence BR1 [D0, §0.2.4].

Le modèle *logique* est scindé en différentes vues pour permettre une meilleure lisibilité :

- les vues globales regroupent les entités par fonctions et par dépendances,
- une vue du modèle centré sur les entités de référence partagées (référentiel géographique, monétaire, etc.).
- une vue du modèle centré sur les entités *Clients*,
- une vue du modèle centré sur les entités *Marchands*,
- une vue du modèle centré sur les entités *Transactions*,
- une vue du modèle centré sur les entités dédiées à la sécurité et à la conformité réglementaire,

Points particuliers du modèle

Le modèle transactionnel est conçu en priorité pour permettre une écriture rapide des transactions. La journalisation des lectures et des écritures est conçue pour ne pas ralentir le chemin critique du paiement (cf. [D1] - sujet d'intégration OLTP/NoSQL et OLTP/CDC). La même contrainte est appliquée sur le scoring de fraude (cf. [D1] - sujet d'intégration OLTP/Scoring).

Les autres fonctionnalités prises en compte n'interfèrent pas directement avec la procédure de paiement :

- gestion des alertes de sécurité,
- gestion des demandes de modification et des consentements,
- gestion des revue de fraude,
- gestion des abonnements,
- gestion des catalogue des marchands,
- gestion des plans de tarification des marchands,
- gestion des remboursements, litiges et rétro-facturations.

Limites du modèle

Le domaine fonctionnel du modèle de données est limité à la première phase du traitement des transactions financières (cf. [D0, §0.4]). La gestion des transactions avec les banques, des dépôts et des reversements aux marchands, ainsi que la plupart des produits et des services adjacents proposés par *Stripe*, sont hors du périmètre du modèle.

Le modèle ne partitionne aucune table et ne prévoit pas de mise en cache. Ces choix, ainsi que la préparation d'un éventuel sharding par marchand, sont justifiés en [D2-C, §1.C.4]. Les stratégies d'indexation sont définies dans le modèle physique [D2-B].

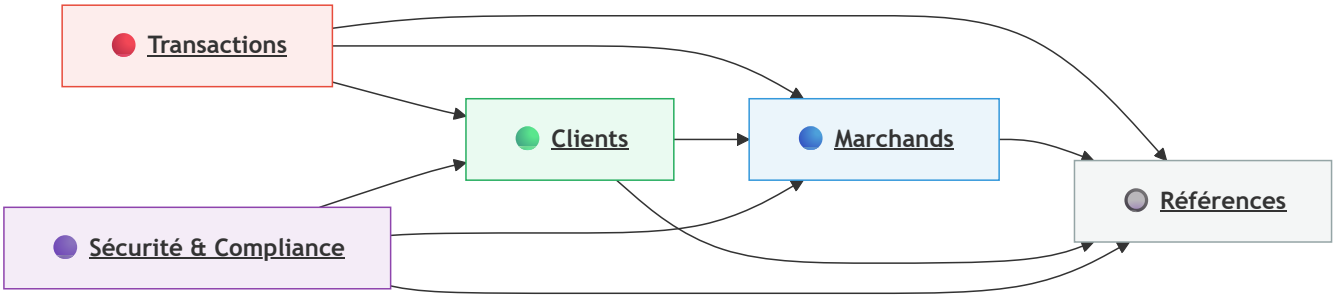
D2.2. Vues globales

D2.2.1. Groupes fonctionnels et dépendances

Cette vue regroupe les entités du modèle par fonctions et par dépendances :

- *Références* :
  - tables autonomes d'énumération référencées par tous les autres groupes ;
- *Marchands* :
  - enregistrement des marchands et des tables dépendantes (produits, etc.),
  - ne dépend d'aucun autre groupe que celui des références ;
- *Clients* :
  - enregistrement des clients et des tables dépendantes (abonnement, etc.) ;
  - s'appuie sur le groupe des marchands ;
- *Transactions* :
  - enregistrement des transactions et des tables dépendantes (remboursement, fraudes, etc.) ;
  - s'appuie à la fois sur le groupe des marchands et celui des clients ;
- *Sécurité & Conformité* :
  - journaux et enregistrements diverses répondant aux exigences de sécurité (incidents, etc.) et de conformité réglementaire (consentement, demande de modification, etc.) ;
  - dépend des clients et des marchands (mais pas des transactions) ;

Noter que les vues suivantes reprennent les codes couleurs représentés ci-dessous.



> *Légende : Une flèche du groupe A vers le groupe B signifie que des tables de A référencent ou utilisent des tables de B.*

Noter que les vues des sections suivantes reprennent les codes couleurs représentés ci-dessus.

D2.2.2. Groupes et tables

Cette vue reprend les groupes fonctionnels précédents en détaillant les tables qu'ils contiennent.
Les tables sont estampillées en fonction de leur nature, de leurs dépendances, et des contraintes d'écriture imposées (cf. Règles de gestion dans [D2-A]).

Les tables et leurs dépendances sont détaillées dans les vues suivantes.



> *Etiquettes de classe :*
>
> - ★ : Entité métier principale ;
> - ⊞ : Entité métier secondaire (extension directe ou indirecte d'une entité principale) ;
> - ☷ : Enumération ;
> - ▤ : Journal ;
> - 🔒 : Entité immuable (ne peut être ni modifiée ni supprimée après création) ;
> - ✕ : ne peut pas être journalisé (dans les journaux des accès et des écritures) ;

Points particuliers

- *Pas de partitionnement :* les tables `Transaction`, `DataAccessLog`, `ChangeAuditLog` et `FraudScoringLog` restent des tables ordinaires, à clé primaire *simple*. Les tables dépendantes de `Transaction` ne portent aucune colonne de partition et leurs clés étrangères sont définies par `transaction_id` seul.

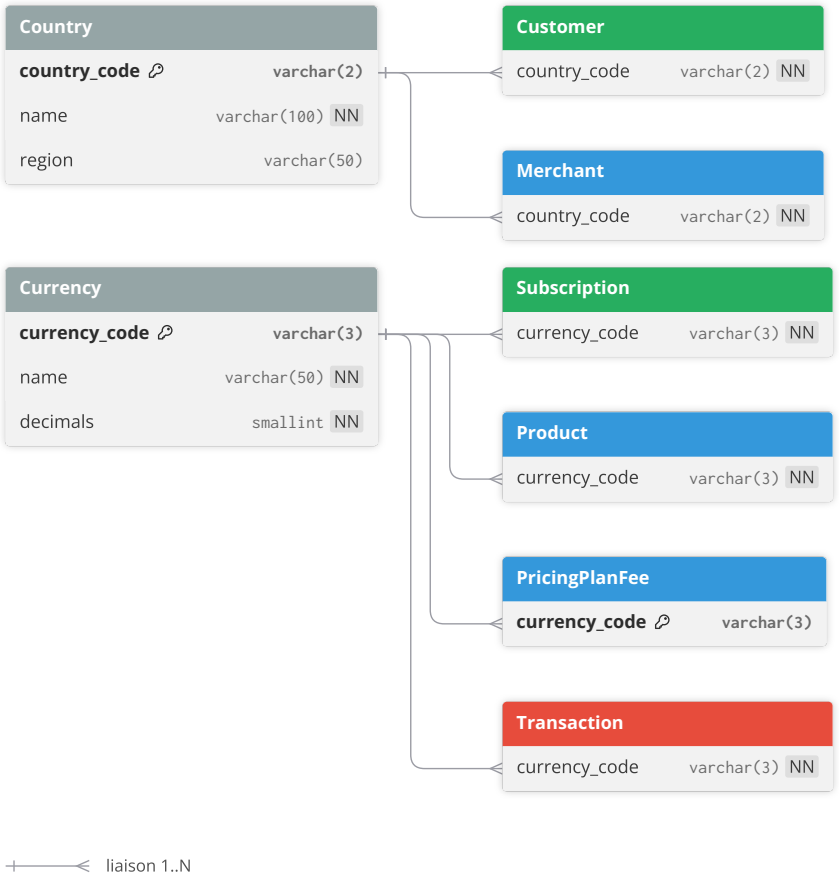
D2.3. Vues *Références*

Les tables de listes de référence partagées sont les suivantes (code couleur 🔘) forment deux sous-groupes :

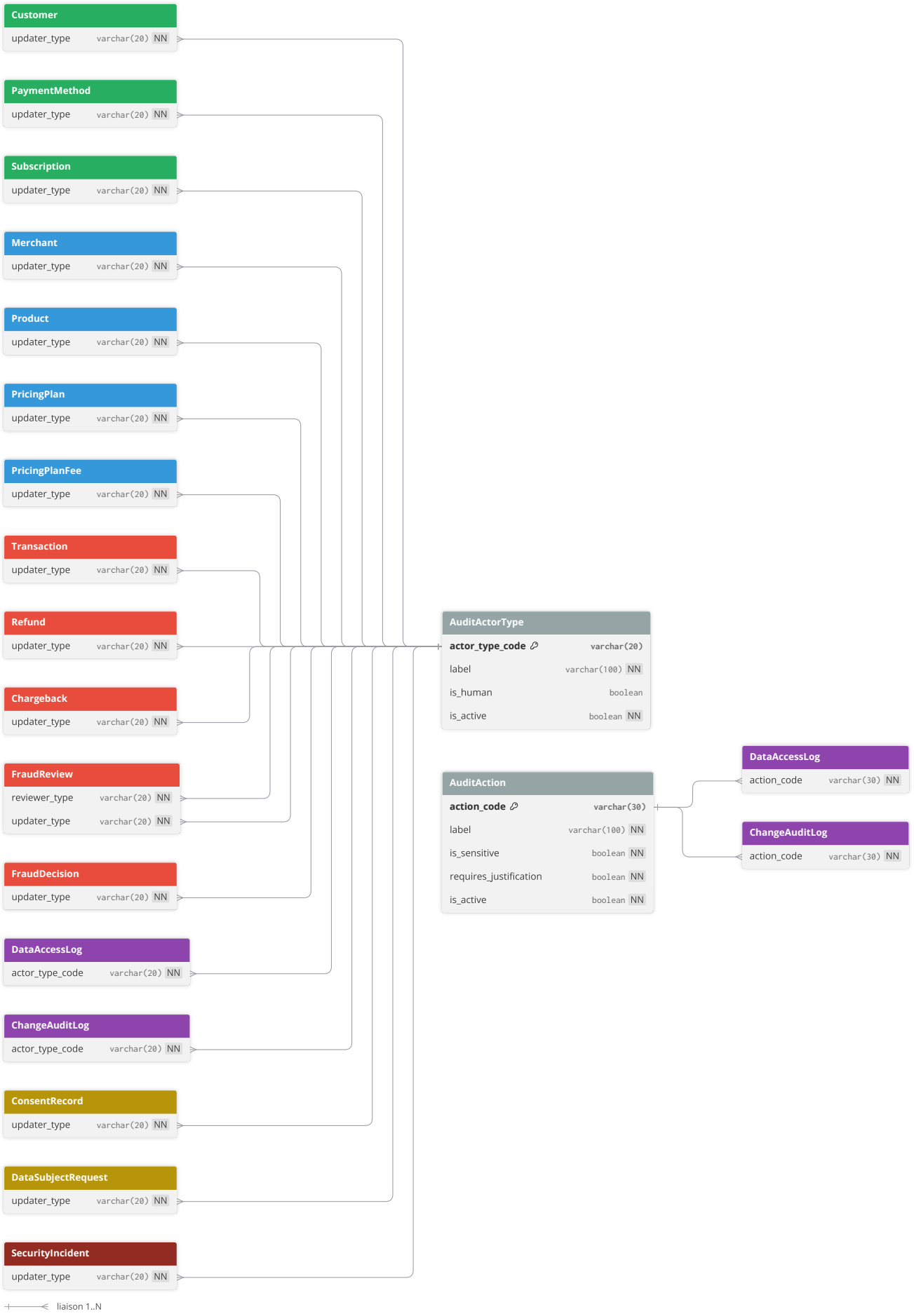
- listes de référence pour le coeur de métier : `Country`, `Currency` ;
- catégories d'auditables pour la sécurité et la conformité : `AuditActorType`, `AuditAction` ;
  - Types des acteurs auditables :
    - user,
    - system,
    - service,
    - merchant_api ;
  - Types des actions auditables :
    - Actions de lecture : read, export, login,
    - Actions d'écriture : create, update, delete, grant, revoke ;

Remarque : la plupart des listes de référence du modèle restent dans leur groupe d'utilisation tant qu'elles ne sont pas partagées par d'autres groupes (ex: *Regulation*, *Card Network*, etc.). Exceptionnellement, la table `AuditAction` suit l'autre table d'audit, `AuditActorType`, dans le groupe des références alors qu'elle n'est utilisée que par les tables de journalisation du groupe *Sécurité & Conformité*.

Coeur de métier

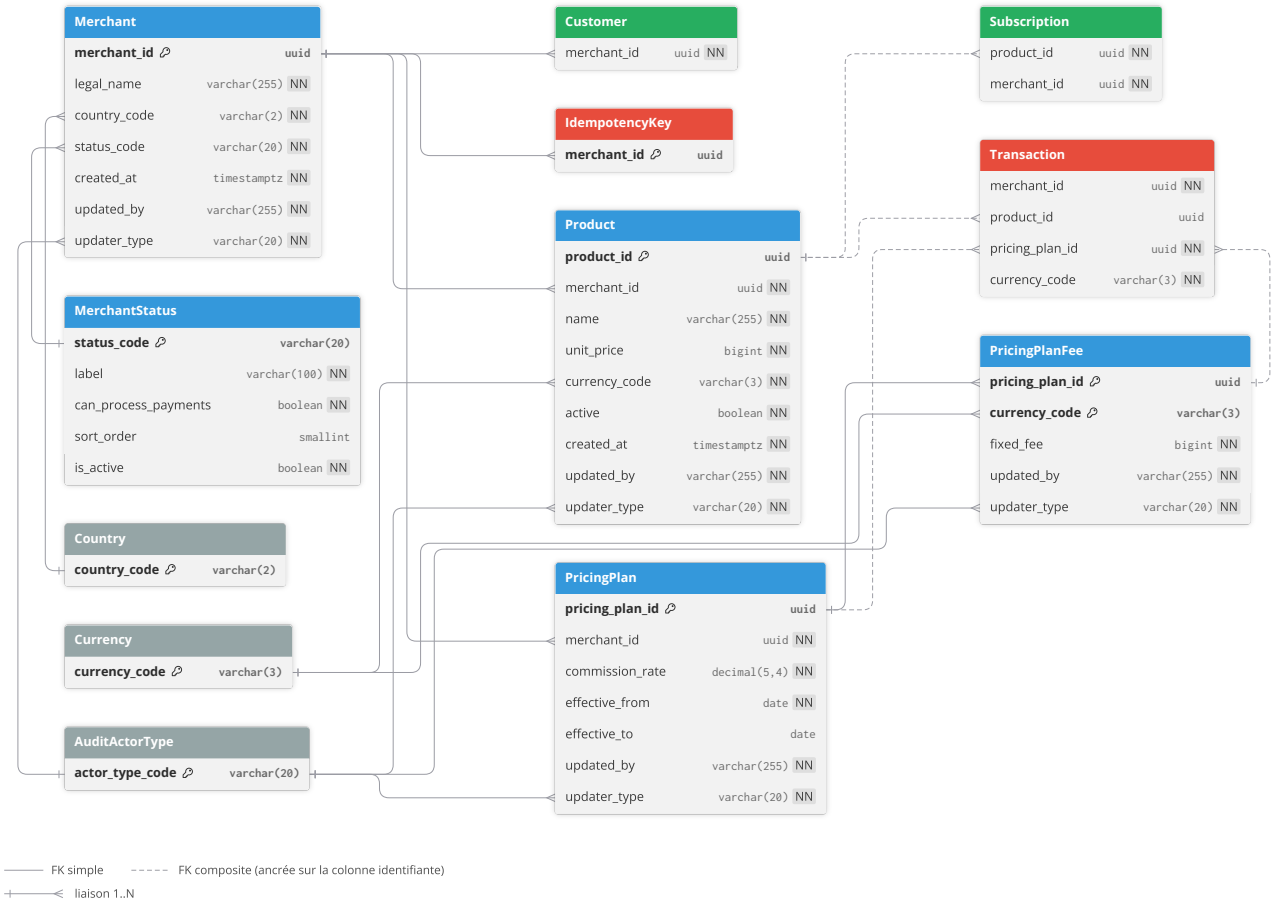


Catégories d'auditables



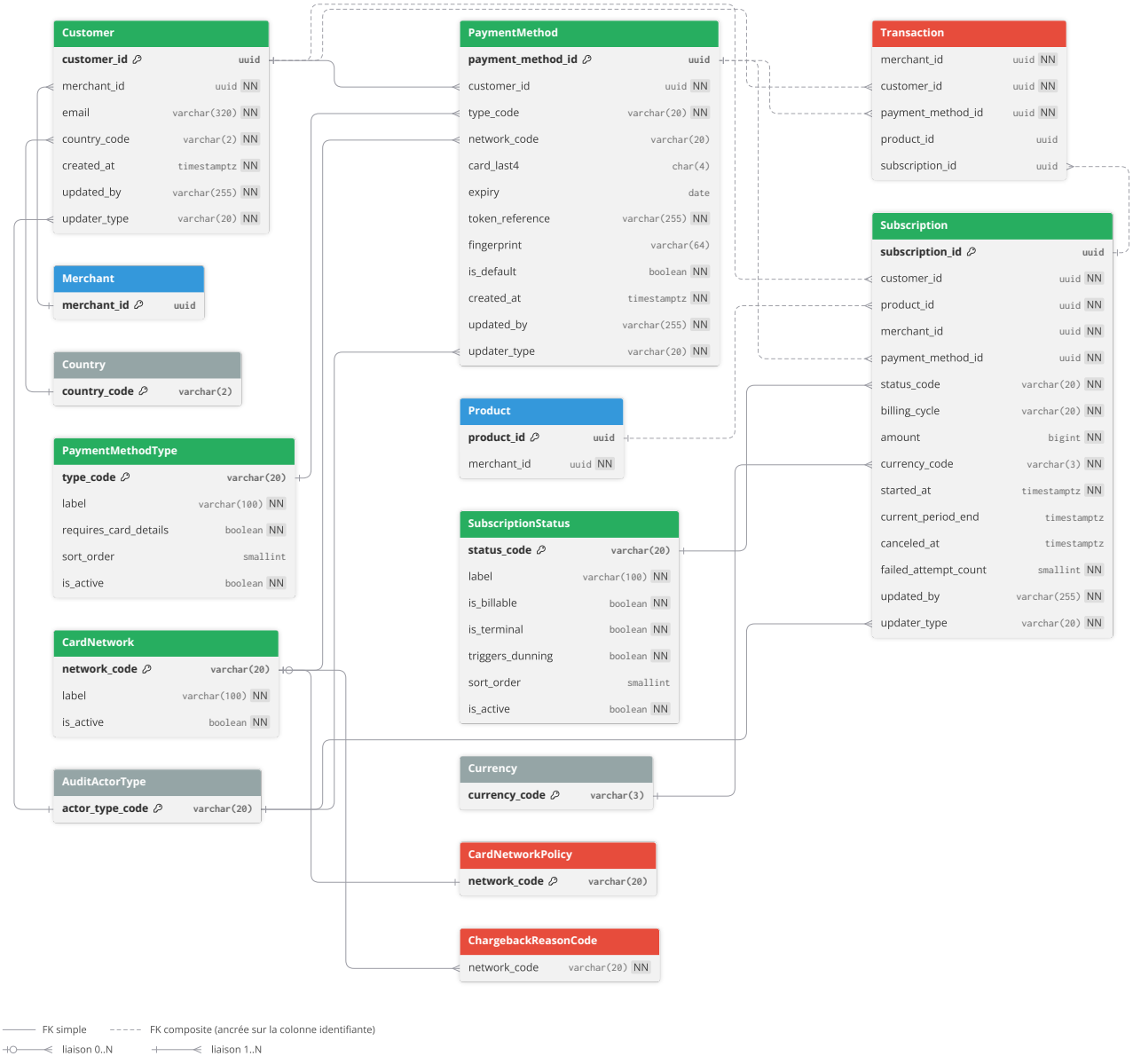
D2.4. Vue *Marchands*

Les tables de l'écosystème *marchand* sont les suivantes (code couleur 🔵) :



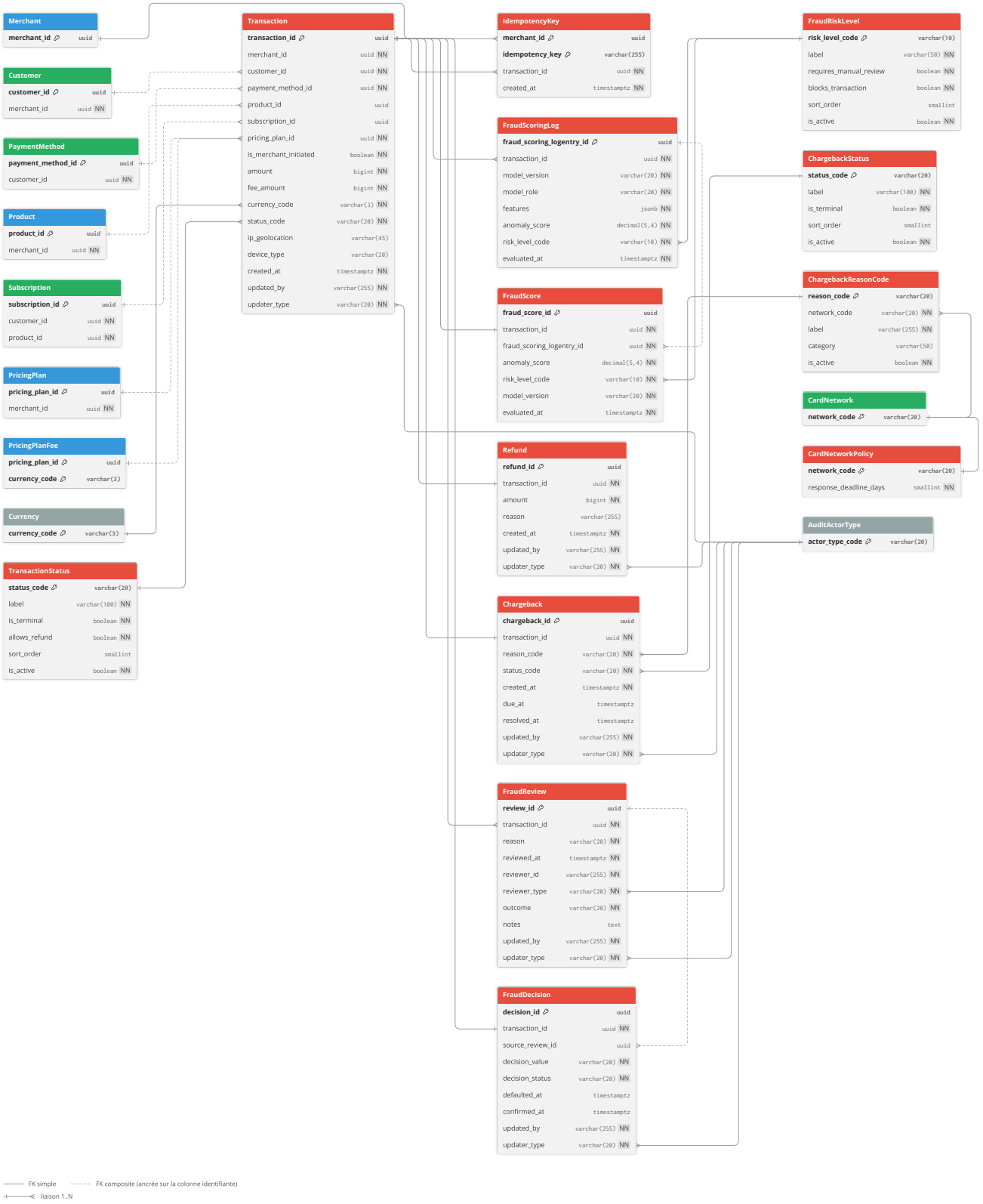
D2.5. Vue *Clients*

Les tables de l'écosystème *client* sont les suivantes (code couleur 🟢) :



D2.6. Vue *Transactions*

Les tables de l'écosystème des *transactions* sont les suivantes (code couleur 🔴) :



Remarque #1 : le délai de réponse du marchand (`response_deadline_days`) est spécifique au réseau de cartes et non au code de justification du litige (`ChargebackReasonCode`). Dans le cadre d'une normalisation stricte, il aurait fallu découpler le réseau de cartes lui même (ex: `CardNetworkPolicy`) du le code de justification, de la même manière que l'on a sorti `Regulation` de `DataSubjectRequest` .

Remarque #2 : la date limite de réponse du marchand (`due_at`) devrait être précalculée pour `Chargeback` , comme elle l'est pour `DataSubjectRequest` : `due_at = created_at + response_deadline_days` .

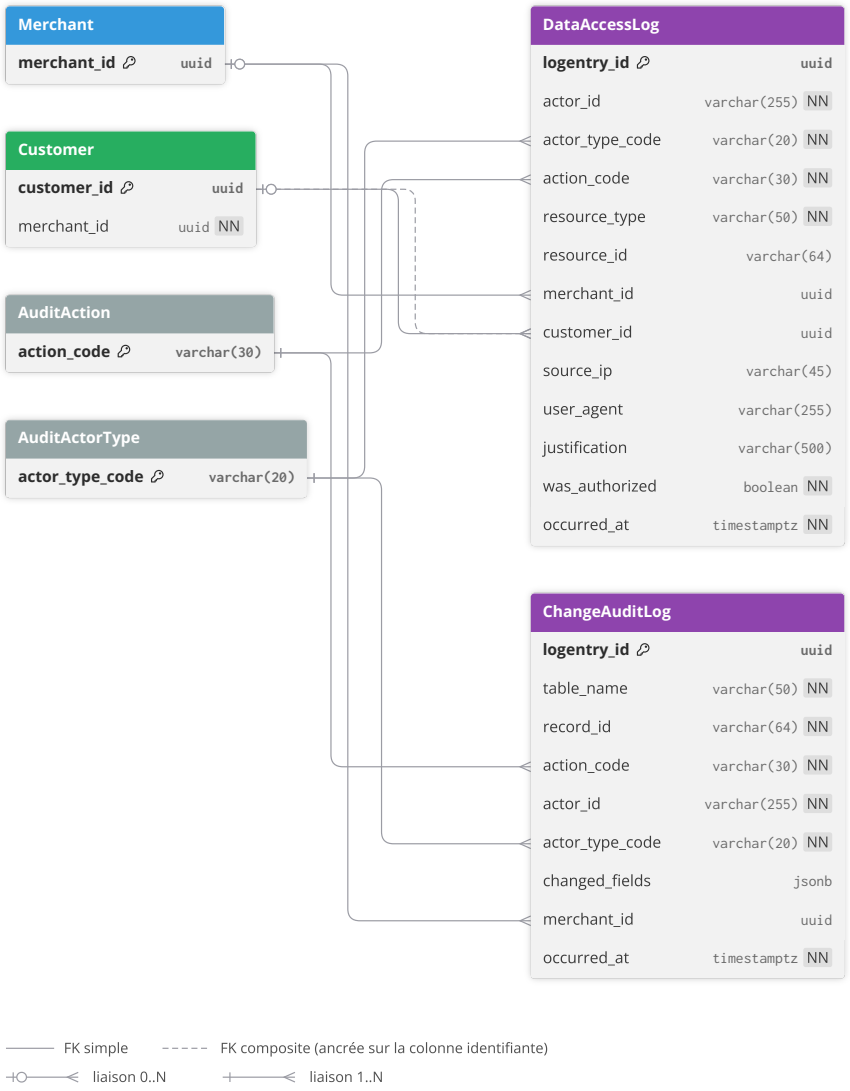
D2.7. Vue *Sécurité & Conformité*

Les tables dédiées aux questions de *sécurité* et de *conformité* sont les suivantes (codes couleurs 🟣, 🟡 et 🟠) :

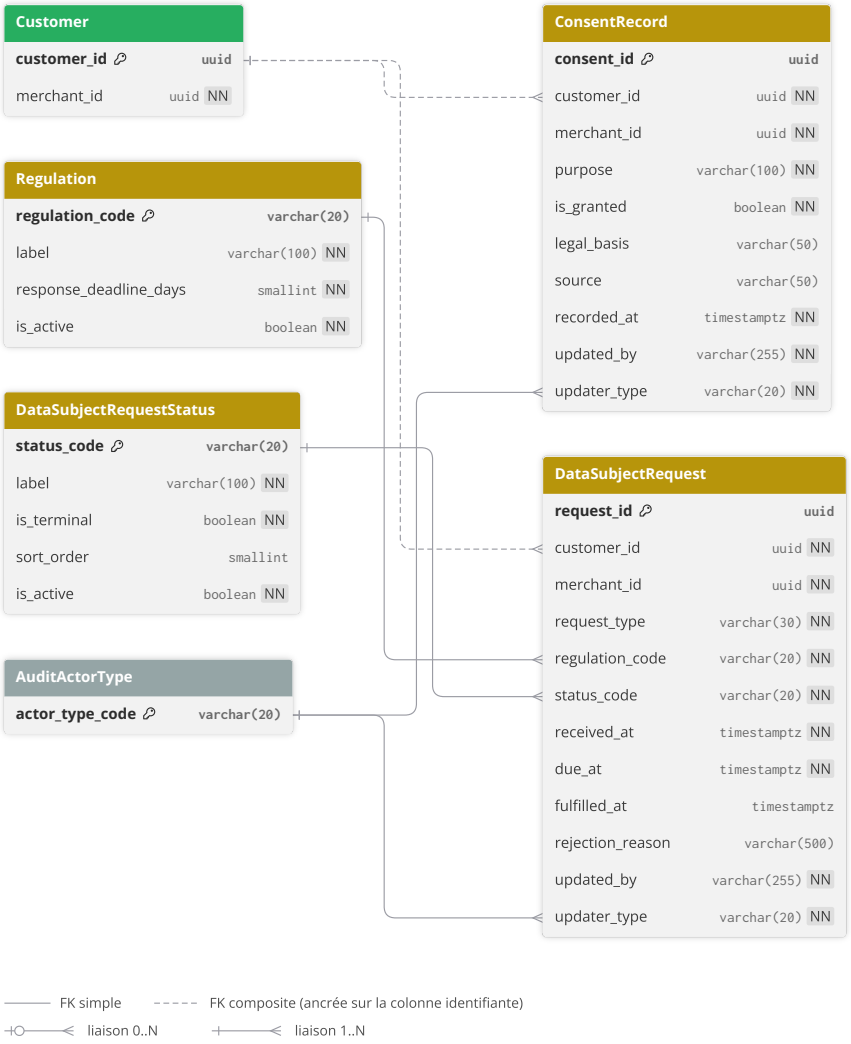
Les tables dédiées aux questions de *sécurité* et de *conformité* forment les sous-groupes suivants :

- Journalisation des accès et des écritures (🟣)
  - > concerne à la fois la sécurité et la conformité réglementaire ;
- Conformité et droits des utilisateurs (🟡)
  - > demandes de modification ou de suppression, consentements ;
- Incidents de sécurité (🟠)
  - > incidents de sécurité.

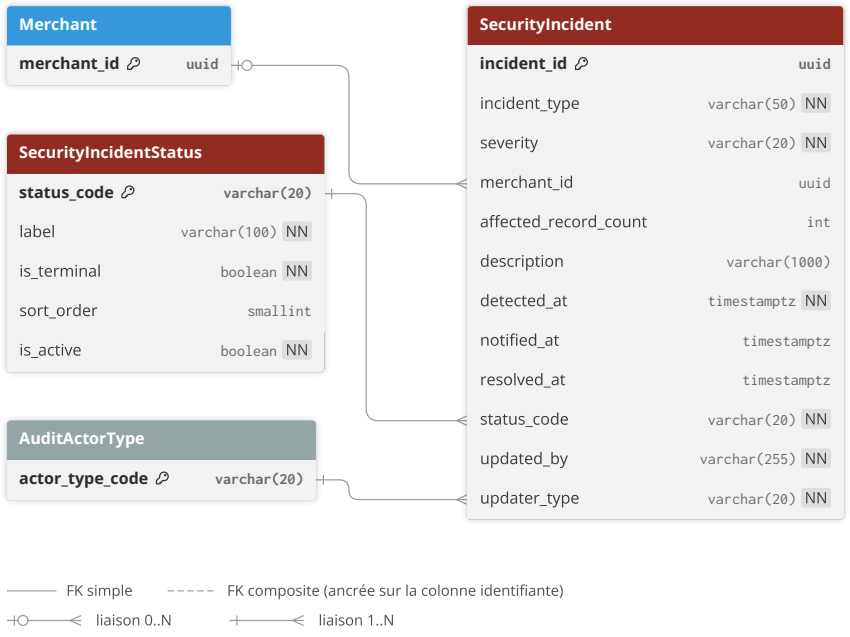
Journalisation des accès et écritures



Conformité et droits des utilisateurs



Incidents de sécurité



Couverture des exigences

D'après [D0, §0.2.8] (prend en compte les annexes A, B, C et D).

#	Fonctionnalité	A/	C/	Sources
F01.01	Volume transactionnel élevé	(TR3)	🔴	Index ciblés [D2-B] ; `Transaction` auto-suffisante et entités dépendantes découplées ([D2-C] §1.C.1.2, §1.C.1.3) ; un seul primaire en écriture ([D2-D] §1.D.1) ; partitionnement et *sharding* écartés ([D2-C] §1.C.4).
F01.02	Latence transactionnelle minimale	(TR3)	🟢	`Transaction` auto-suffisante ([D2-C] §1.C.1.2), index ciblés et partiels [D2-B], journalisation et scoring hors chemin critique (§D2.1, [D2-C] §1.C.1.7) ; surcoût de la réplication synchrone intra-région ([D2-D] §1.D.2.2).
F01.03	Cohérence et intégrité transactionnelle (ACID)	**D2**	🟢	Natif SGBD ; clés, contraintes d'intégrité et contraintes différées, journaux immuables [D2-B] ; règles de gestion [D2-A] §1.A.4.
F01.04	Schémas normalisés	**D2**	🟢	3NF ([D2-C] §1.C.1.2) ; modèle conceptuel [D2-A], modèle logique [D2], modèle physique [D2-B].
F01.05	Transactions multi-documents	(BR1)	🟢	Natif SGBD : transactions multi-tables et contraintes différées (`IdempotencyKey` , `Refund`) ; mécanisme d'idempotence [D2-C] §1.C.1.9.
F01.06	Reprise après sinistre	(BR1)	🔴	[D2-D] §1.D.4 : bascule inter-région manuelle, RPO >> 0, exercice trimestriel ; §1.D.5 : sauvegarde continue et fenêtres de rétention ; l'outillage de la bascule inter-région n'est pas spécifié (§1.D.1).
F01.07	Capacité à accueillir de nouvelles fonctionnalités	(C1)	🟢	3NF, tables de référence pour les énumérés porteurs de métadonnées, entités dépendantes découplées ([D2-C] §1.C.1.3, §1.C.1.5).
F01.08	Paiements, remboursements, rétro-facturations	**D2**	🟢	`Transaction` , `IdempotencyKey` , `Refund` , `Chargeback` ; §D2.6 ; [D2-C] §1.C.1.3 (`Refund` vs `Chargeback`) et §1.C.1.9.
F01.09	Gestion des abonnements	**D2**	🟢	`Subscription` , `Product` , `PricingPlan` versionné ; §D2.5 ; [D2-C] §1.C.1.6 et §1.C.2 ; [D2-A] §1.A.6.1.
F04.01	Intégration OLTP/OLAP/NoSQL	D1, D5	🔴	[D2-B] définit la publication *CDC* vers l'OLAP et le *NoSQL* et en exclut `ChangeAuditLog` pour éviter une boucle d'écriture ; le reste relève de [D1] et [D5].
F04.06	Synchronisation en temps réel	D5	🟢	Réplication physique en streaming, synchrone intra-région ([D2-D] §1.D.2) ; publication *CDC* pour les autres systèmes [D2-B].
F04.09	Cohérence des données entre systèmes	D5	🔴	Part OLTP : publication *CDC* et `ChangeAuditLog` [D2-B] ; la cohérence à terme et la résolution de conflits relèvent de [D5].
F04.13	*Change Data Capture*	D5	🔴	[D2-B] : publication *CDC* (lecture asynchrone du WAL), `updater_type` permettant de dériver `ChangeAuditLog` du flux, exclusion de `ChangeAuditLog` de la publication ; le transport relève de [D5].
F05.01	Montée en charge horizontale	(TR3)	🔴	Écartée par [D0] §0.4.3 pour l'OLTP ; le modèle la prépare sans la mettre en œuvre : UUID v7 générées sans point central et cloisonnement par marchand ([D2-C] §1.C.4).
F05.02	Haute disponibilité	(TR3)	🔴	[D2-D] §1.D.2 et §1.D.3 : réplication synchrone, bascule automatique par *Patroni* et *etcd*, protections contre le *split-brain*, fenêtre de bascule, proxy de connexion, supervision ; haute disponibilité locale seulement, le système reste centralisé.
F05.03	Bases de données distribuées	(TR3)	🔴	Écartées par [D0] §0.4.3 (SGBD centralisé, cible *PostgreSQL*) ; [D2-D] §1.D.3.9 décrit des répliques asynchrones inter-régions, en lecture et en secours, non une base distribuée.
F05.04	Sharding	(TR3)	🔴	Préparé sans être mis en œuvre : clés UUID sans point central, cloisonnement par marchand, clé de répartition envisagée `hash(merchant_id)` ([D2-C] §1.C.4) ; la stratégie de répartition n'est pas formalisée.
F05.05	Partitionnement	(TR3)	🔴	Écarté et justifié : la clé primaire d'une table partitionnée devient composite et se propage à toutes les tables filles ([D2-C] §1.C.4, §D2.2.2) ; recommandation portée hors périmètre dans [D2-B].
F05.07	Indexation	D4	🟢	Index ciblés et partiels par table [D2-B] ; index de lot et index temporels pour les balayages et la purge de rétention ([D2-C] §1.C.4).
F06.03	Contrôles d'accès (RBAC)	D6	🔴	Rôles et privilèges par fonction (application, écriture des journaux, lecture seule) et journaux immuables [D2-B] ; le RBAC applicatif relève de [D6].
F06.04	Journalisation d'audit	D6	🟢	`DataAccessLog` et `ChangeAuditLog` en écriture seule, `AuditAction` , `AuditActorType` ; §D2.7 ; [D2-C] §1.C.1.7 (écriture asynchrone) et

#	Fonctionnalité	A/	C/	Sources
				§1.C.3 (séparation des deux journaux).
F06.05	Surveillance de l'utilisation des données	(C5)	🟢	`DataAccessLog` (qui, quoi, quand, autorisé ou non), `AuditAction`, `AuditActorType`, `SecurityIncident` ; [D2-C] §1.C.3.
F06.06	Surveillance en temps réel des failles de sécurité	D6	🟠	`SecurityIncident` enregistre les incidents, y compris ceux de l'infrastructure ([D2-D] §1.D.4.3) ; [D2-D] §1.D.3.8 couvre la supervision et les alertes du SGBD ; la détection applicative en temps réel n'est pas dans le modèle.
F06.07	Cadre de sécurité et outils de surveillance	D6	🟠	[D2-C] §1.C.3 justifie le modèle de sécurité et de conformité ; [D2-D] §1.D.3.8 décrit supervision et alertes ; le plan d'ensemble relève de [D6].
F07.01	Conformité réglementaire du stockage et du traitement	D6	🟢	`Regulation`, `DataSubjectRequest`, `ConsentRecord`, `SecurityIncident` ; délais matérialisés comme données mesurables ([D2-C] §1.C.1.7) ; §D2.7.
F07.02	RGPD	(TR5)	🟢	Droits de modification et de suppression, consentements, découplage de `Regulation` ([D2-C] §1.C.3) ; double rattachement de `ConsentRecord` (§1.C.1.7).
F07.03	PCI-DSS	(TR5)	🟢	Le PAN reste hors système : `last4`, `token_reference`, `fingerprint` ([D2-C] §1.C.1.8).
F07.04	CCPA	(C5)	🟠	`Regulation` porte le délai de réponse (45 jours) et `DataSubjectRequest` s'y rattache ; aucune fonction propre au CCPA.
F07.05	Pluralité des juridictions	(BR5)	🟠	`Regulation` modélise plusieurs réglementations et leurs délais propres ([D2-C] §1.C.3) ; ni localisation ni résidence des données.
F07.06	Contrôles de conformité automatisés	D6	🟠	Délais matérialisés (`due_at`) et contraintes [D2-B] ; l'exécution des contrôles et l'alerting relèvent de [D6].
F07.07	Reporting de conformité	D6	🟠	Les données nécessaires sont disponibles et datées (`DataSubjectRequest`, `ConsentRecord`, `SecurityIncident`) ; la production des rapports relève de [D6] et de l'OLAP.
F08.01	Modèles de détection de fraude en temps réel	D4, D7, D8	🟠	`FraudScore`, `FraudScoringLog`, `FraudReview`, `FraudDecision` ([D2-A] §1.A.6.7, [D2-C] §1.C.1.3) ; scoring hors chemin critique (§D2.1) ; le scoring lui-même relève de [D1] et [D7].
F09.01	Conception des modèles de données par système	D1, **D2**, D3, D4	🟢	[D2] pour le modèle logique, [D2-A] pour le modèle conceptuel et les règles de gestion, [D2-B] pour le modèle physique, [D2-C] pour les justifications, [D2-D] pour la réplication et la reprise.

> *Légende : « A/ » : les livrables qui couvrent la fonctionnalité dans D0, ou, à défaut, l'exigence technique, l'exigence d'affaire ou l'enjeu source ; « C/ » : niveau de couverture par D2 ;* 🟢 *: fonctionnalité traitée ;* 🟠 *: fonctionnalité traitée partiellement (réserve est indiquée dans la colonne Sources).*

Références

Spécifications

- [D0] Stripe Project Introduction
-

Livrables

- [D1] Data Architecture

Annexes

- [D2-A] OLTP Data Conceptual Model
- [D2-B] OLTP Data Physical Model
- [D2-C] OLTP Design Notes
- [D2-D] OLTP Replication, Failover and Disaster Recovery Strategy